

more connections. You can compile the *udp-server.hs* and *udp-client.hs* files using the following commands:

```
$ ghc --make udp-server.hs
[1 of 1] Compiling Main      ( udp-server.hs, udp-
server.o )
Linking udp-server ...

$ ghc --make udp-client.hs
[1 of 1] Compiling Main      ( udp-client.hs, udp-
client.o )
Linking udp-client ...
```

You can start the UDP server in one terminal:

```
$ ./udp-server
Server started ...
```

You can then run the UDP client in another terminal:

```
$ ./tcp-client
```

You will now see the ‘Hello, world!’ message is printed in the terminal where the server is running:

```
$ ./udp-server
Server started ...
Hello, world!
```

The *network-uri* module has many useful URI (uniform resource identifier) parsing and test functions. You can install them on Fedora using the following command:

```
$ cabal install network-uri
```

The *parseURI* function takes a string and attempts to convert it into a URI. It returns ‘Nothing’ if the input is not a valid URI, and returns the URI, otherwise. For example:

```
ghci> :m + Network.URI

ghci> parseURI "http://www.shakthimaan.com"
Just http://www.shakthimaan.com

ghci> parseURI "shakthimaan.com"
Nothing
```

The type signature of the *parseURI* function is given below:

```
ghci> :t parseURI
parseURI :: String -> Maybe URI
```

A number of functions are available for testing the input URI as illustrated in the following examples:

```
ghci> isURI "shakthimaan.com"
False

ghci> isURI "http://www.shakthimaan.com"
True

ghci> isRelativeReference "http://shakthimaan.com"
False

ghci> isRelativeReference "../about.html"
True

ghci> isAbsoluteURI "http://www.shakthimaan.com"
True

ghci> isAbsoluteURI "shakthimaan.com"
False

ghci> isIPv4address "192.168.100.2"
True

ghci> isIPv6address "2001:0db8:0a0b:12f0:0000:0000:0000:0001"
True

ghci> isIPv6address "192.168.100.2"
False

ghci> isIPv4address "2001:0db8:0a0b:12f0:0000:0000:0000:0001"
False
```

The type signatures of the above functions are as follows:

```
ghci> :t isURI
isURI :: String -> Bool

ghci> :t isRelativeReference
isRelativeReference :: String -> Bool

ghci> :t isAbsoluteURI
isAbsoluteURI :: String -> Bool

ghci> :t isIPv4address
isIPv4address :: String -> Bool

ghci> :t isIPv6address
isIPv6address :: String -> Bool
```

You can make a GET request for a URL and retrieve its contents. For example:

```
import Network
```